Deterministic AST-Guided Fault Localization for 4-Bit Quantized Developer Agents

Muhammad Khoiruzzadittaqwa
Institut Al Bahjah, Cirebon, Indonesia
Email: muhzadit@gmail.com

Abstract

Feeding entire source files into 4-bit quantized language models causes rapid fault localization failure. While W4A16 weight quantization compresses model memory to fit single consumer GPUs, it flattens the dynamic range of self-attention logits over long context windows. When an agent ingests full source modules, accumulated quantization noise disperses attention across the sequence, inflating softmax entropy and obscuring defect origins. In this study, we decouple symbol localization from code editing using deterministic Abstract Syntax Tree (AST) skeletonization within the Google Agent Development Kit (ADK). By stripping implementation bodies and exposing only class hierarchies, method signatures, decorators, and docstrings, our pipeline cuts inspection payloads by 77.97% (from 3,981 to 877 mean tokens). Benchmarked across 60 real defect instances on Kaggle L4x4 GPU nodes running 4-bit Gemma 31B, our method lifts defect resolution from 48.33% to 85.00%, drops patch syntax failures from 23.33% to 3.33%, and cuts mean latency from 63.79 to 15.53 seconds per issue. These empirical findings demonstrate that aggressive syntactic pruning restores discrimination in low-bit attention mechanisms.

1. Introduction

When software repair agents operate on unquantized frontier models in cloud clusters, supplying generous multi-file context is standard practice. The Gemma 4 Developer Agent Competition changes those operating assumptions: agents must run a 4-bit quantized 31B parameter model (*gemma-4-31b-it-qat-w4a16-ct*) strictly offline inside a single Kaggle L4 GPU container, with a 12-hour total runtime envelope.

In our early experiments, supplying complete source files to this 4-bit model produced an immediate and severe failure mode: attention dilution. As context length expanded beyond 3,000 tokens, rounding noise in the 4-bit projection matrices accumulated across attention dot products. Softmax distributions flattened toward high-entropy uniformity. Rather than attending to the specific function that triggered an exception, the model scattered its weights across surrounding utility code. Consequently, the agent misidentified innocent helper functions as culprits, hallucinated arguments, and proposed bloated multi-function edits that broke working unit tests.

This observation prompted our central question: *Can deterministic syntactic pruning preserve attention sharpness in 4-bit quantized models without sacrificing fault localization recall?*

We show that it can. We designed a three-stage repair pipeline on the declarative Google Agent Development Kit (ADK) that replaces raw source text with tree-sitter AST skeletons during search. By isolating structural symbol definitions from concrete function logic, the model reasons about system topology under a bounded token budget. Our primary contributions are:

1. A theoretical model linking token volume to softmax entropy degradation in 4-bit quantized attention heads.
2. A declarative three-stage harness (Localizer, Synthesizer, Verifier) enforcing strict token caps and automated syntax verification.
3. An empirical evaluation on 60 real defect instances showing a 77.97% token reduction, an increase in repair success from 48.33% to 85.00%, and a fourfold throughput gain on Kaggle L4 hardware.

2. Theoretical Formulation: Quantization and Attention Entropy

In multi-head self-attention, head h computes its attention matrix over sequence length N as:

$$A_h = \text{softmax}\left(\frac{Q_h K_h^T}{\sqrt{d_k}}\right)$$

where $Q_h, K_h \in \mathbb{R}^{N \times d_k}$. Under uniform 4-bit weight quantization with 16-bit activations (W4A16), coordinate quantization introduces a noise tensor $E = \widehat{W} - W$. This perturbation propagates into the pre-softmax attention logits:

$$S_h = \frac{\big(X(W_Q + E_Q)\big)\big(X(W_K + E_K)\big)^T}{\sqrt{d_k}} = S_h^{\text{exact}} + \Delta S_h$$

Because quantization errors across independent channels act as zero-mean random perturbations, logit error variance scales linearly with sequence length:

$$\text{Var}(\Delta S_h) \propto N \cdot \sigma_E^2$$

As N scales from 1,000 tokens to 8,000 tokens, logit perturbation variance widens. For a query token q_i attempting to attend to a sparse defect token k_j, the signal-to-noise ratio degrades. We measure this dispersion across the sequence using normalized Shannon entropy:

$$H\big(A_{h(i,:)}\big) = -\frac{1}{\log N} \sum_{j=1}^{N} A_h(i,j) \log A_h(i,j)$$

When $H \approx 1$, the attention distribution approaches a flat uniform distribution. The model loses its capacity to resolve discrete code lines. Restricting input context to $N < 1500$ tokens bounds this perturbation variance, preserving low entropy ($H < 0.35$) and restoring symbol discrimination.

3. Declarative Three-Stage Architecture

Our system runs within the Google ADK execution environment, enforcing separation of duties across three sub-agents.

Sub-Agent	Sampling Temp	Token Ceiling	Core Responsibility
Localizer	0.10	1,500 tokens	AST skeleton parsing, symbol extraction, candidate ranking.
Synthesizer	0.20	2,048 tokens	Minimal Unified Diff generation for suspect lines.
Verifier	0.05	512 tokens	Git apply validation and 80-line churn ceiling enforcement.

3.1 Localizer: AST Skeleton Search

The Localizer inspects file trees without reading function bodies. An AST parser converts Python source files into skeleton representations containing class declarations, method signatures, decorators, and docstrings. If an issue mentions `AttributeError in DataFrame.merge`, the Localizer queries the AST symbol registry for class `DataFrame` and method `merge`, identifying line bounds in fewer than 900 tokens while omitting intermediate statements.

3.2 Synthesizer: Minimal Diff Generation

Once line bounds are isolated, the Synthesizer retrieves only the target lines with a 15-line margin. Guided by Strunk's Rule 13 (omit needless words), the prompt enforces minimal diff authoring. Formatting refactors, variable renames, and gratuitous line reordering are rejected by prompt contract.

3.3 Verifier: Automated Guardrails

The Verifier acts as an automated sanity gate by running `git apply --check` against each patch. If a patch exceeds 80 lines of churn or modifies more than two files, the Verifier rejects the patch. In our experiments, 4-bit model

outputs exceeding 80 lines for a single bug were uniformly hallucinatory; rejecting them prevented cascading test failures.

4. Empirical Evaluation & Ablation Study

We benchmarked the pipeline against 60 defect instances extracted from open-source scientific Python repositories (including pandas, scikit-learn, and sympy). All evaluations ran offline on Kaggle L4x4 GPU nodes running `gemma-4-31b-it-qat-w4a16-ct`.

Evaluation Metric	Full-Context	AST Skeleton	Absolute Delta	Relative Change
Mean Input Tokens	3,981	877	−3,104	−77.97%
Defect Resolution Rate	48.33%	85.00%	+36.67%	+75.87%
Mean Latency per Issue	63.79 s	15.53 s	−48.26 s	−75.65%
Patch Application Errors	23.33%	3.33%	−20.00%	−85.73%
Cumulative 12-Hour Capacity	677 issues	2,781 issues	+2,104 issues	+310.78%

The empirical results match our theoretical derivation. Ingesting full source modules overloaded the model's 4-bit attention layers, producing a 23.33% syntax failure rate and capping repairs at 48.33%. AST skeletonization kept input context within 877 mean tokens, lifting defect resolution to 85.00% while reducing latency to 15.53 seconds per issue. This speed allows the agent to evaluate up to 2,781 candidate issues within the competition's 12-hour cumulative limit.

5. Failure Modes and Threats to Validity

The 15.0% unresolved instances stemmed from two limitations:

1. *Dynamic Metaclass Tricks*: When libraries inject attributes at runtime via metaclass hooks, static AST parsers fail to extract definitions, forcing fallback to broad search.
2. *High Cross-Module Coupling*: Fixes requiring coordinated edits across four or more files exceed our two-file safety cap.

6. Conclusion

Deterministic AST skeletonization resolves attention dilution in 4-bit quantized language models. By pruning 77.97% of context noise, our Google ADK pipeline achieved an 85.00% repair rate on Gemma 31B while staying well within Kaggle's 12-hour execution envelope.